

ground-score: an AI support agent for @AmazonHelp, and how much to trust it

Kavin Nandha · Hiver SDE intern take-home · September 2026
github.com/kavinnandha/ground-score · full decision log in DECISIONS.md

Everything in Results and Failure analysis is produced into results/ by make reproduce, which runs with the API key stripped from the environment so any uncached step fails loudly rather than quietly making a live call.

1. Problem framing

What "good" means for this brand

@AmazonHelp is a public, high-volume, logistics-heavy queue. Most inbound is a status question rather than a problem to solve, and the brand's own reply is usually a short apology plus one instruction or link. It rarely hides behind DMs but it does hand off: 10.8% of first replies send the customer elsewhere (0.8% to DMs, 9.9% to a contact page), low enough that "draft a grounded reply" is a real task, high enough that a system imitating this brand learns to hand off about one time in ten. And the expensive mistake is public and permanent: a needless escalation costs an agent thirty seconds, a wrong auto-reply is a tweet from a brand account, and the cases where it goes wrong (fraud, damage, a customer already let down twice) are the cases where the customer can least absorb it.

So "good" means: correct routing matters far more than correct replies; grounded beats fluent; escalations must be attributable to a named rule, not a vibe; and the headline should be an operating point — *X% auto-handled at no more than Y% false-auto* — rather than an accuracy figure. Section 3.3 reports that on this data no such operating point exists, which is the main negative result here.

What I deliberately didn't build

Multi-turn dialogue (the agent acts on the first customer message only). **Live tool use** — no order lookup, no account access, which is the single biggest cap on the ceiling here, because a large share of these tickets cannot be answered without account data and no prompt work fixes that; recognising those and escalating is the correct behaviour. **PII redaction** (PII is detected and used as an escalation trigger, not scrubbed). **Fine-tuning. Multilingual support** — non-Latin-script messages are filtered out of the corpus; they exist in the queue and this system does not serve them. **Serving:** no API, no queue consumer, no latency budget.

2. Method

Stage	Approach
Brand choice	7 brands profiled on handoff rate, reply substance, intent diversity, multi-turn rate. AmazonHelp wins on lexical diversity (0.183, highest) and multi-turn rate (0.597, highest) at a 10.8% handoff rate
Corpus	8,496 threads from an 11,000 sample, stable-hash split into 7,998 retrieval history / 498 evaluation pool held out of the index entirely
Taxonomy	KMeans over embedded inbound → LLM names each cluster → hand-merged into 12 classes, 5 of them never-auto
Golden set	150 examples, 3 strata (60% traffic-proportional, 25% rare, 15% hard), dev 70 / test 80 by stable hash assigned before labelling. Manually adjudicated against the taxonomy, overriding the weak label on 31.3%
Label quality	50-row blind second pass by a <i>different</i> model: intent κ 0.579, action κ 0.109 on badly skewed marginals
Agent	retrieve(k=5) → classify → draft (≤280 chars, must cite precedent) → route. Deterministic rules run before the LLM gate so every escalation carries a rule name
Evaluation	Bootstrap 95% CIs, coverage-vs-harm curve, LLM judge validated against blind reference scores
Models	qwen3:4b local classifies and drafts; gemini-3.5-flash-lite judges (different vendor and family, so the drafter is not grading itself); nomic-embed-text embeds

My pre-registered guess for the brand was SpotifyCares. The data rejected it at 37.7% handoff. The first version of that profile was also wrong: it measured only DM-style deflection and put AmazonHelp at 0.008. Reading actual retrieved replies showed the brand says "please reach out to us here" rather than "DM us". Fixing the metric moved it to 0.108, a 13× undercount. The ranking survived, but it stood on a wrong number until somebody checked it against raw text.

3. Results

3.1 The brand's own reply, scored by the same judge

For all 70 dev rows, @AmazonHelp's actual 2017 reply went through the same blind judge, on the same message, against the same retrieved precedent as the agent's draft. One model scored all 140 calls; providers.switches is empty, so this is one judge and not two averaged. This is the one genuinely human comparison in the evaluation, it needs no labels, and that is why it leads.

dimension	agent	brand's own reply	delta	comparable?
groundedness	4.39	3.64	+0.74	no — see below
resolution	4.23	3.53	+0.70	yes
tone fit	4.76	3.94	+0.81	yes
safety	4.53	3.90	+0.63	yes
comparable mean	4.50	3.79	+0.72	
would_send	0.87	0.54	+0.33	

Read literally, a 4B model on my desktop writes better support replies than the people who answered these tickets. I don't believe that for a second, and working out why is worth more than the table. Our gate would have blocked **32 of the 70 replies this brand actually posted (46%)**.

Groundedness is excluded from the comparable mean structurally: the human's claims rest on the order, the account and the tracking page, none of which the judge can see, so the rubric scores them as unsupported. The other three are contaminated too, in three measurable ways. **The judge marks down replies it was handed incomplete** — seven historical replies end "(1/2)" because the brand splits long answers, and they score 3.00 against 3.86 for the rest. **It punishes correct support behaviour it cannot verify** — four replies telling a customer to stop posting tracking details publicly average 2.58, and on one the judge wrote the reply "invents an ungrounded policy" and "would embarrass the brand", when `route.py` hard-escalates public PII for exactly the reason that human agent gave. **It rewards problem-shaped boilerplate** — on thread 2750418 a customer tweets a pleased photo of a successful Prime Now delivery, the human replied in kind, the agent replied "*I'm sorry you haven't received your Prime delivery to Orlando*", and the judge scored the agent 3.33 higher.

So the honest headline of this table is not that the agent is better. It is that an LLM judge with no account access and no thread context systematically prefers grounded-sounding boilerplate to real support work, and every reply-quality number in this report inherits that bias.

3.2 Against the baselines

Dev split, $n = 70$, bootstrap 95% CIs. Fitted baselines are scored out-of-fold because dev is their fit split; in-sample `simple_tfidf_nn` returns 1.000 by memorisation.

system	intent acc	macro F1	coverage	false-auto	esc. recall
trivial_always_auto	0.171 [0.09–0.26]	0.024	1.000	0.700 [0.59–0.80]	0.000
trivial_always_escalate	0.171	0.024	0.000	—	1.000
simple_tfidf_nn	0.314 [0.21–0.43]	0.250	0.000	—	1.000
agent_no_retrieval (ablation)	0.700 [0.60–0.80]	0.720	0.000	—	1.000
agent	0.729 [0.63–0.83]	0.733	0.443	0.613 [0.44–0.78]	0.612

The classification result is real and the routing result is not. Intent accuracy goes $0.171 \rightarrow 0.314 \rightarrow 0.729$ with non-overlapping CIs, on a 12-class problem. Routing is the opposite: the agent auto-handles 44% of dev and **61% of what it auto-sent should have gone to a human**. The always-auto baseline is wrong 70% of the time. Being wrong 61% of the time instead is not a system anybody should deploy, and quoting 44% coverage as a headline would be the most misleading thing in this report.

The ablation says grounding does almost nothing for classification and almost everything for the reply. Intent accuracy moves $0.700 \rightarrow 0.729$, inside the CI — no established difference. Counting draft-level defects by pattern rather than by the judge, so that a judge blind spot cannot hide them:

system	draft defects / 70 rows	worst category
agent	7	3 public PII solicitations, 2 unfilled placeholders, 2 wrong names
simple_tfidf_nn	10	10 replies greet the customer by a name from a <i>different</i> thread
agent_no_retrieval	45	32 public PII solicitations, 7 invented deadlines, 4 unfilled [link] placeholders
trivial_always_auto	0	one canned string: it cannot err because it never says anything

Thirty-two times in seventy rows the ungrounded drafter asked a customer to post an order number, an account email or a **payment method** on a public timeline — the exact thing this brand's own replies exist to warn against. With retrieval on, three. Blind reference scores agree: comparable mean 3.83 against 2.52, would-send 85% against 20%. So grounding does not make the model understand the customer better. It stops it inventing a procedure. That is not the answer the intent-accuracy column gives.

Three of the four zero-coverage rows are uninformative: `trivial_always_escalate` is zero by construction, and the other two are zero because `draft_ungrounded` and `low_retrieval_similarity` fire on everything they produce, so their false-auto column is empty for want of a denominator rather than because they are safe.

3.3 The operating point does not exist

Swept on dev over both available signals with the deterministic rules held fixed:

signal	tau	coverage	false-auto
confidence	0.000–0.825	0.629	0.614
confidence	0.850–0.925	0.614	0.628
similarity	0.725	0.529	0.568
similarity	0.800	0.229	0.625
similarity	0.850	0.100	0.857

False-auto never drops below 57% at any threshold that automates anything, on either signal, and on similarity it gets worse as you tighten. The budget was set at 10% before the sweep ran. Nothing comes close, so `configs/thresholds.yaml` ships conservative defaults carrying `budget_feasible: false` and the harm floor recorded next to them. The confidence curve is flat because the signal is constant: the drafter self-reports 0.95 on **every one of the 19 false-auto rows**. It is not a weak probability estimate; it is not a probability.

The first version of the tuner hid this. `best_threshold` treated "automate nothing" as a feasible operating point — its false-auto rate is defined as 0.0 for want of a denominator — so it returned coverage 0.0 and the config read like a successful fit at `tau_confidence: 0.95`. Fixed; the tuner now reports infeasibility instead of returning it.

3.4 Judge validation

$n = 60$ paired replies across four systems. Reference scores are blind and were collected before the judge ran.

dimension	Spearman ρ	weighted κ	reference	judge	bias
groundedness	+0.663	+0.665	3.52	3.73	+0.22
resolution	−0.075	−0.018	2.33	3.88	+1.55
tone fit	+0.724	+0.670	3.55	4.03	+0.48
safety	+0.665	+0.744	3.75	3.97	+0.22
would_send (gate)	—	+0.541	sends 43%	sends 57%	raw 0.767

Three of four dimensions land at moderate agreement with a small positive bias, and the gate — the judgement routing depends on — comes in at κ 0.541. Enough to report reply quality with a caveat; not enough to report it to two decimals.

Resolution is the exception and it is not noise, it is a specific disagreement. $\rho -0.075$ means the judge's resolution score carries no information about the reference score, while sitting 1.55 points higher. One rubric rule causes it: a handoff scores 1 on resolution, on purpose, so that imitating this brand's 10% deflection rate is not rewarded. The judge does not apply it. Across the agent's own replies it scores handoff-shaped ones **4.44** against 4.20 for everything else, and on *"Pls reach out here so we can check this for you"* it gave resolution **5**, justifying it as "provides an actionable next step". The judge is scoring a different, more permissive rubric that it brought with it, and the one dimension where the written rubric deliberately departs from the training distribution is exactly where it refuses to follow. Every resolution figure above, including the +0.70 in 3.1, inherits that.

Until this run every one of those κ s printed nan: `metrics.agreement` built its arrays with `dtype=object`, sklearn infers that as target type "unknown" and raises on every call, and the except beneath returned NaN with a comment blaming a degenerate rater. A NaN in a results table reads as "small sample", not as "this has never worked".

3.5 The held-out test split, scored once

$n = 80$. `results/test_runs.jsonl` is the audit trail for "scored once", and it has more than one line in it, so: every entry records `providers.live_calls`, and **exactly one entry has a non-empty one** — 39 hosted calls, the live scoring run. Every other line is a `make reproduce` replay of identical inputs from the committed cache, showing `live_calls: {}`. The split was *scored* once and *replayed* as often as anybody runs the reproduction, and the file lets you tell those apart rather than asking you to take a count on trust.

system	intent acc	macro F1	coverage	false-auto	esc. recall
trivial_always_auto	0.075 [0.03–0.14]	0.012	1.000	0.688 [0.57–0.79]	0.000
simple_tfidf_nn	0.475 [0.36–0.59]	0.403	0.000	—	1.000
agent_no_retrieval	0.637 [0.53–0.74]	0.621	0.000	—	1.000
agent	0.650 [0.54–0.75]	0.664	0.425	0.471 [0.30–0.63]	0.709

Same shape as dev and weaker in every direction, which is what an honest test split is for. **The agent still beats both baselines and by less than dev suggested** — the gap over TF-IDF narrows from 41 points to 17 and the CIs now overlap, so "the agent beats TF-IDF" is supported and "by how much" is not. (Part of that narrowing is mechanical: on dev the fitted baseline is scored out-of-fold, on test it is fitted on all 70 dev rows.) **Routing is less bad and still not deployable:** false-auto 0.471 at unchanged coverage, against the always-auto baseline's 0.688. **The ablation gap disappears entirely** — 0.650 against 0.637 — so across both splits the conclusion is that retrieval does not improve intent classification at all, and earns its place solely through reply quality and draft safety.

The test reply scores are excluded, with the reason recorded rather than hidden. Partway through judging, the hosted judge died on a socket error and the role chain failed over to local `gemma3:4b` as designed, so `judge_model` reports **38 rows scored by gemini-3.5-flash-lite and 42 by gemma3:4b**. Averaging two judges into one number would be meaningless, so reply quality stays a dev result where one model scored every row. Worth noting the chain did not fire on quota, the anticipated cause — 39 of 500 daily calls

had been used. It fired on an ordinary network fault, which is the more common thing and the one a single-provider setup would have turned into a dead run.

4. Failure analysis

Ranked by frequency out of `results/failure_analysis_dev.json`, not picked by hand. Dev, 70 rows, 31 auto-sent, 19 of those wrong.

1. Nothing detects an angry customer (6 of 19 false-autos). The largest single cause. Six messages were auto-answered where the label says a human must take it because the customer is angry or distressed, and there is **no rule of any kind** for that state: the deterministic layer checks PII, legal, fraud, safety and never-auto classes, and anger is none of those. On *"so it's acceptable to send parcels with NO packaging now is it?! Well done for ruining Christmas presents! I'm so angry."* (1464242) the agent auto-sent a returns link. **Hypothesis:** a missing rule, not a model failure — the classifier is *right* on 4 of these 6 rows, because intent and emotional state are orthogonal and only one is modelled. Also the cheapest fix available: the golden set already carries `angry_or_distressed` tags to train on.

2. other is a hole in the taxonomy, and a never-auto hole (5 of 19). `other` is the largest golden class (19 of 150) and the classifier misses it into an auto-able class six times on dev. Every one is a genuine request the taxonomy has no box for: when a pre-order is charged ($\rightarrow$ `delivery_late`), a review that was never published ($\rightarrow$ `promotional_or_off_topic`), receiving someone else's entire order ($\rightarrow$ `delivery_not_received`). **Hypothesis:** a miss *out of* a never-auto class removes the only protection those messages have, and **11 of the 19 intent errors are in that direction**. The bucket is not miscellaneous either — it clusters into pre-sales, missing-item-from-order, seller/KDP and contact-channel failures, so three real classes would empty most of it. That makes it a taxonomy bug, not a classifier bug. The clearest single failure in the set is thread 13563: asked *when* a pre-order is charged, the agent publicly auto-sent **"We aim to process payments 24 hours after your order ships"**, a specific policy claim supported by no precedent and as far as I can tell invented.

3. The confidence signal is constant, so a third of the design does nothing. On all 19 false-auto rows the self-reported confidence is **0.95** — not clustered high, identical. `tau_confidence` is a live parameter, documented and configured, that has never made a decision. **Hypothesis:** asking a 4B model for a probability returns a fluency signal formatted as a number.

4. PII detection reads text; customers post screenshots. Thread 420919 escalates in the golden set on `contains_personal_data` and was auto-answered: the regexes are correct and found nothing because the tracking details are in an attached image — which is also why the brand's own reply to that thread was a privacy warning. Same shape at 816343 and 2619718. **Hypothesis:** not fixable with a better pattern. Roughly one dev row in twenty carries its content in an image, which is a hard ceiling on text-only routing. The right response is a rule — a message whose text is mostly a URL or a hashtag escalates on insufficient content — and it does not exist yet.

5. Removing a wrong safety escalation bought no coverage at all. The `safety_or_harm` rule matched a bare `\bfire\b`. On an Amazon queue that is a keyword trap: measured over the corpus it matches 103 messages and **92 (89%) are Fire TV, Fire HD or Fire tablet questions**. It hard-escalated three ordinary device questions on dev as safety incidents. It is fixed and tested, and **coverage did not move** — all three now escalate on `low_retrieval_similarity` instead. That non-result is the interesting part twice over: the escalation rules are redundant enough that fixing one changes only which reason gets recorded, so the value is entirely in auditability, which is a stated design goal — a system that tells a support lead "safety incident" about a Fire TV question is lying to its operator even when the routing is right. And it previews what a second brand would find: a rule correct in general and wrong for one brand's vocabulary, which reading the code never catches and one measurement does.

Also counted: draft defects found by pattern rather than by the judge, since the judge rates several of these as sendable. On the agent's 70 dev rows: 3 replies ask for an order number or account email on a public timeline, 2 ship an unfilled placeholder (one literally *"I'm so sorry about this, [Customer Name]!"*), 2 greet the customer by a name lifted from a different thread. Seven in seventy, none needing a model to detect, all blockable by a regex in CI.

5. What is misleading about my headline number?

1. One human annotator is still a limitation. I manually adjudicated all 150 golden examples and wrote the blind reply-quality reference scores before seeing any LLM-judge output. That makes the reported judge agreement judge-versus-human, but one person's routing policy is not a replacement for a support organisation's policy. The weak labeller was `qwen3:4b`; I overrode it on **47 of 150 rows (31.3%)**. Reviewing proposals can still introduce anchoring, so the override rate is reported rather than treated as proof the labels are unbiased.

1b. The optional second pass is human-versus-model. A blind `gemma3:4b` pass over 50 rows gives intent κ **0.579** (raw 0.620) and action κ **0.109** (raw 0.680). The second pass escalates 94% of rows against my 66%, so this is a model sensitivity check, not human inter-annotator reliability.

2. The golden set was pre-labelled by the system it evaluates. Adjudicating a proposal is not labelling cold, and no amount of care removes the anchoring.

3. n = 80 on test, so intervals are wide. Differences under roughly 10 points between systems are not distinguishable from noise and should be read as "not established".

4. The pooled number overstates real-traffic accuracy, the opposite of what I expected. I wrote that over-sampling rare and hard cases must drag the number down. Measured, it goes the other way: `proportional` (real traffic mix, $n=39$) scores **0.667**, `rare` ($n=20$) scores **0.900**, `adversarial` ($n=11$) scores 0.636, pooled 0.729. **Quote 0.667 for expected traffic; the headline is six points high, not**

low. The cause is the sampling design: the rare stratum is drawn by inverse *cluster* frequency, deliberately, so it cannot be circular with the classifier — but small clusters are not hard intents, and in practice it filled with packaging complaints and promotional chatter, two of the easiest classes. A stratum designed to stress the tail is inflating the average instead.

5. Historical replies are what the brand *did*, not what was *good* — and the rule meant to handle that is the one the judge ignores (3.4). The safeguard against rewarding imitation is written down and not enforced.

6. The escalation labels are a written policy applied by a model, not Amazon's policy. So false-auto rate measures disagreement with my stated policy, not tweets that harmed anybody. Read the 61% with that in front of it.

7. The judge is a different family from the drafter, but still a small model, and it is validated against another model rather than a person.

8. The hard-case slice was designed by the same person who built the agent. It uses model-independent surface heuristics to limit that, and the blind second annotator agreed with only 20% of its intent labels against 63% on the proportional stratum — so whatever else it is, it is genuinely the hard end.

9. The thresholds are not fitted at all, because the fit failed (3.3). This section previously said "fitted on ~70 dev examples", which was wrong.

10. The drafter is a small cheap model, chosen because ~1000 calls per full run exceeds every free hosted budget available to me. Comparisons *between* systems stay fair since every system uses the same generator; the absolute reply scores are not a statement about the design.

11. The retrieval backend almost entirely determines what the agent sees. Two embedding backends over the same corpus and the same 120 queries agree on the single most similar precedent **7.5%** of the time, and their top-5 sets overlap **11.3%**. "Grounded in how the brand historically resolved this" means grounded in whatever neighbours the encoder happens to surface. Both return plausible precedent on inspection, which is what makes this easy to miss: spot-checking retrieval and finding it fine does not establish that it is stable.

12. One brand, one snapshot. AmazonHelp in late 2017. Nothing here establishes that the taxonomy, the thresholds or the judge's behaviour transfer.

6. What I'd do with one more week

Ranked by how much they would change what I currently believe. The first is not close.

- 1. Put a person on the labels.** Every number rests on labels a model wrote, and that is the thing most likely to be wrong in a way nothing downstream can detect. One person re-adjudicates all 150 rows in `tools/label_cli.py`; a second does 100 so the κ becomes real inter-annotator agreement between humans. That also turns judge validation back into judge-versus-human. Nothing else needs new code.
- 2. Build a router signal that separates harm.** Neither confidence nor similarity does (3.3), and six of 19 false-autos are distressed customers with no rule covering them. Train a distress classifier on the golden `angry_or_distressed` and `needs_account_access` tags, put it ahead of the LLM gate, score it on the same curve. If that does not move the curve, the honest conclusion is that this task needs account access rather than a better model, and the system should escalate by default.
- 3. Fix the other hole.** Three new classes — pre-sales, missing-item-from-order, seller/KDP — would empty most of the bucket and remove the second largest source of false-autos.
- 4. Human preference over pairwise comparisons.** Absolute 1–5 scores are the least reliable part of the reply evaluation, and 3.1 shows the judge preferring grounded-sounding boilerplate to real support work.
- 5. Transfer test on hulu_support** without re-tuning anything. Whatever degrades is what was overfitted to AmazonHelp; the fire bug is a preview.
- 6. Cost and latency accounting.** Three LLM calls per message is a real operating cost. Given that the thresholds do no routing work, a rules-only fast path for the easy majority is probably most of the value.